



Saint Columban College
College of Computing Studies
Pagadian City



Vault-Door-1

vault-door-1 



Medium

Reverse Engineering

picoCTF 2019

AUTHOR: MARK E. HAASE

Hints 

Description

This vault uses some complicated arrays! I hope you can make sense of it, special agent. The source code for this vault is here:

[VaultDoor1.java](#)

1

34,559 users solved



62% Liked



picoCTF{d35cr4mbl3_tH3_cH4r4cT3r5_7ffa94}

Submit Flag

Author: The Analyst: Hyposelenia

Challenge: vault-door-1

Category: Reverse Engineering

Date: 4/14/26



I. Objective

The goal of this challenge is to **find the correct password (flag)** by analyzing a program

Instead of guessing the password, we **look at how the program checks the password** and rebuild the correct answer from there.

II. Background

In **Reverse Engineering**, we don't run the program blindly—we **read the code to understand how it works**.

In this challenge:

- The program is written in Java.
- It checks if your input (password) is correct.
- But instead of comparing the password directly, it checks **specific characters at specific positions**.

Think of it like this:

The program says:

“Character #0 must be ‘p’”

“Character #1 must be ‘i’”

“Character #2 must be ‘c’”

...and so on.

So instead of guessing, we just **follow the instructions and rebuild the password ourselves**.

III. Tool Used

- **Visual Studio Code** - to open and read the Java file



IV. Methodology

1. Downloaded the file provided in the challenge

VaultDoor1.java

2. Opened the Java source code using a code editor

```
class VaultDoor1 {
    public boolean checkPassword(String password) {
        return password.length() == 32 &&
            password.charAt(index: 0) == 'd' &&
            password.charAt(index: 29) == 'a' &&
            password.charAt(index: 4) == 'r' &&
            password.charAt(index: 2) == '5' &&
            password.charAt(index: 23) == 'r' &&
            password.charAt(index: 3) == 'c' &&
            password.charAt(index: 17) == '4' &&
            password.charAt(index: 1) == '3' &&
            password.charAt(index: 7) == 'b' &&
            password.charAt(index: 10) == '_' &&
            password.charAt(index: 5) == '4' &&
            password.charAt(index: 9) == '3' &&
            password.charAt(index: 11) == 't' &&
            password.charAt(index: 15) == 'c' &&
            password.charAt(index: 8) == 'l' &&
            password.charAt(index: 12) == 'H' &&
            password.charAt(index: 20) == 'c' &&
            password.charAt(index: 14) == '_' &&
            password.charAt(index: 6) == 'm' &&
            password.charAt(index: 24) == '5' &&
            password.charAt(index: 18) == 'r' &&
            password.charAt(index: 13) == '3' &&
            password.charAt(index: 19) == '4' &&
            password.charAt(index: 21) == 'T' &&
            password.charAt(index: 16) == 'H' &&
            password.charAt(index: 27) == 'f' &&
            password.charAt(index: 30) == '9' &&
            password.charAt(index: 25) == '_' &&
            password.charAt(index: 22) == '3' &&
            password.charAt(index: 28) == 'f' &&
    }
}
```

3. Looked for the function that checks the password
4. Found multiple lines like:
password.charAt(0) == 'p'
password.charAt(1) == 'i'
5. Noticed that each line tells us **what character goes in each position**
6. Carefully arranged all characters in the correct order
7. Combined them to form the full flag

V. Result

picoCTF{d35cr4mbl3_tH3_cH4r4cT3r5_7ffa94}



```
return password.length() == 32 &&  
  
|  
  
picoCTF{d35cr4mb13_th3_ch4r4cT3r5_7ffa94}
```

VI. Explanation

The program does something like this:

```
if (password.charAt(0) == 'p' &&  
    password.charAt(1) == 'i' &&  
    password.charAt(2) == 'c') {
```

This means:

- Position 0 = 'p'
- Position 1 = 'i'
- Position 2 = 'c'

So we start building:

p i c _ _ _ _ ...

Then more lines appear:

```
password.charAt(3) == 'o'  
password.charAt(4) == 'C'
```

Now it becomes:

p i c o C _ _ ...

We continue this process **until all positions are filled**.

Important idea:

- `charAt(x)` = “character at position x”
- We are basically solving a **puzzle where each clue fills one letter**

Once all letters are placed correctly, we get the full flag.



VII. Conclusion

This challenge teaches a very important lesson:

You don't always need to guess the answer—you can read the code and let it tell you the answer.

Key takeaways:

- Reverse engineering = understanding how a program works
- Programs often reveal secrets if you read them carefully
- Breaking a problem into small parts makes it easy to solve

— **The Analyst: Hyposelenia**